BEE 4750 Lab 3: Linear Programming with JuMP

Name: Muhammad Islam

ID: mai58

> **Due Date**
>
> Thursday, 10/23/24, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

In [1]:
```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
  Activating project at `~/Desktop/lab3-mai58-11`
   Installed IrrationalConstants ─ v0.2.6
   Installed HiGHS_jll ──────────── v1.11.0+1
   Installed LoggingExtras ──────── v1.2.0
   Installed SpecialFunctions ───── v2.6.1
   Installed JSON ───────────────── v1.2.0
   Installed MutableArithmetics ─── v1.6.6
   Installed HTTP ───────────────── v1.10.19
   Installed MathOptInterface ───── v1.46.0
   Installed libaom_jll ─────────── v3.13.1+0
   Installed Expat_jll ──────────── v2.7.3+0
   Installed Compat ─────────────── v4.18.1
   Installed BenchmarkTools ─────── v1.6.2
   Installed JSON3 ──────────────── v1.14.3
   Installed CodecBzip2 ─────────── v0.8.5
   Installed HiGHS ──────────────── v1.19.3
   Installed Qt6Base_jll ────────── v6.8.2+2
   Installed Qt6Wayland_jll ─────── v6.8.2+2
   Installed OpenSSL_jll ────────── v3.5.4+0
   Installed MathOptIIS ─────────── v0.1.1
   Installed StructUtils ────────── v2.5.1
   Installed ForwardDiff ────────── v1.2.2
   Installed JuMP ───────────────── v1.29.1
Precompiling project...
    626.0 ms  ✓ OpenSSL_jll
    543.2 ms  ✓ Compat
    611.4 ms  ✓ libaom_jll
    545.0 ms  ✓ LoggingExtras
    637.9 ms  ✓ StructUtils
    998.7 ms  ✓ IrrationalConstants
    476.2 ms  ✓ Expat_jll
    511.4 ms  ✓ CodecBzip2
    537.8 ms  ✓ HiGHS_jll
    508.9 ms  ✓ Compat → CompatLinearAlgebraExt
    866.4 ms  ✓ LogExpFunctions
   1324.2 ms  ✓ Fontconfig_jll
   2603.7 ms  ✓ OpenSSL
   1856.3 ms  ✓ SpecialFunctions
   1394.4 ms  ✓ Qt6Base_jll
   2931.5 ms  ✓ JSON
    748.3 ms  ✓ Cairo_jll
   4015.1 ms  ✓ MutableArithmetics
   2212.1 ms  ✓ StatsBase
    467.8 ms  ✓ DiffRules
    935.0 ms  ✓ ColorVectorSpace → SpecialFunctionsExt
    998.8 ms  ✓ HarfBuzz_jll
   1767.1 ms  ✓ BenchmarkTools
   5254.3 ms  ✓ JSON3
    894.5 ms  ✓ libass_jll
    913.1 ms  ✓ Pango_jll
   3080.4 ms  ✓ ForwardDiff
   1828.9 ms  ✓ FFMPEG_jll
   4001.9 ms  ✓ Qt6ShaderTools_jll
    982.1 ms  ✓ FFMPEG
   1372.9 ms  ✓ GR_jll
   1432.5 ms  ✓ Qt6Declarative_jll
```

```
    307.1 ms  ✓ Qt6Wayland_jll
  10379.4 ms  ✓ HTTP
   2324.2 ms  ✓ GR
  24898.0 ms  ✓ MathOptInterface
   1642.1 ms  ✓ MathOptIIS
   6352.7 ms  ✓ HiGHS
  11179.5 ms  ✓ JuMP
  35924.8 ms  ✓ Plots
  40 dependencies successfully precompiled in 53 seconds. 157 already precom
piled.
```

In [2]:
```
using JuMP # optimization modeling syntax
using HiGHS # optimization solver
```

Overview

In this lab, you will write and solve a resource allocation example using `JuMP.jl`. `JuMP.jl` provides an intuitive syntax for writing, solving, and querying optimization problems.

`JuMP` requires the loading of a solver. [Each supported solver works for certain classes of problems, and some are open source while others require a commercial license]. We will use the `HiGHS` solver, which is open source and works for linear, mixed integer linear, and quadratic programs.

In this lab we will walk through the steps involved in coding a linear program in HiGHS, solving it, and querying the solution.

Exercise (3 points)

Your task is to decide how much lumber to produce to maximize profit from wood sales. You can purchase wood from a managed forest, which consists of spruce (320,000 bf) and fir (720,000 bf). Spruce costs $\$0.12$ per bf to purchase and fir costs $\$0.08$ per bf.

At the lumber mill, wood can be turned into plywood of various grades (see Table 1 for how much wood of each type is required for and the revenue from each grade). Any excess wood is sent to be recycled into particle board, which yields no revenue for the mill.

Plywood Grade	Inputs (bf/bf plywood)	Revenue ($/1000 bf)
1	0.5 (S) + 1.5 (F)	400
2	1.0 (S) + 2.0 (F)	520
3	1.5 (S) + 2.0 (F)	700

Table 1: Wood inputs and revenue by plywood grade. S refers to spruce inputs, F fir inputs.

First, we need to identify our decision variables. While there are several options, we will use G_i, the amount of each grade the mill produces (in 1000 bf).

Using these decision variables, formulate a linear program to maximize the profit of the mill subject to the supply constraints on spruce and fir.

> **JuMP Syntax**
>
> The core pieces of setting up a `JuMP` model involve specifying the model and adding variables, the objective, and constraints. At the most simple level, this syntax looks like this:
>
> ```
> m = Model(HiGHS.Optimizer)
> @variable(m, lb <= x <= ub) # if you do not have upper or
> lower bounds, you can drop those accordingly
> @variable(m, lb <= y <= ub)
> @objective(m, Max, 100x + 250y) # replace Max with Min
> depending on the problem
> @constraint(m, label, 6x + 12y <= 80) # replace "label"
> with some meaningful string you would like to use later
> to query shadow prices, or drop it
> ```
> You can add more constraints or more variables as needed.

> **Using Array Syntax**
>
> You can set up multiple variables or constraints at once using array syntax. For example, the following are equivalent:
>
> ```
> @variable(m, G1 >= 0)
> @variable(m, G2 >= 0)
> @variable(m, G3 >= 0)
> ```
> and
>
> ```
> @variable(m, G[1:3] >= 0)
> ```
> You can also set up multiple constraints using arrays of coefficients and/or bounds. For example:
>
> ```
> I = 1:3
> d = [0; 3; 5]
> @constraint(m, demand[i in I], G[i] >= d[i])
> ```

`JuMP` is finicky about changing objects and constraints, so I recommend setting up all of the model syntax in one notebook cell, which is what we will do here.

In [26]:
```
forest_model = Model(HiGHS.Optimizer) # initialize model object
@variable(forest_model, G[1:3] >= 0) # non-negativity constraints
# Objective: maximize profit (revenue - wood costs)
```

```
@objective(forest_model, Max, 220*G[1] + 240*G[2] + 360*G[3])

# Constraints: spruce and fir supply limits (in 1000 bf)
@constraint(forest_model, spruce, 0.5*G[1] + 1.0*G[2] + 1.5*G[3] <= 320)
@constraint(forest_model, fir,    1.5*G[1] + 2.0*G[2] + 2.0*G[3] <= 720)

print(forest_model)  # show model summary
```

```
Max 220 G[1] + 240 G[2] + 360 G[3]
Subject to
 spruce : 0.5 G[1] + G[2] + 1.5 G[3] ≤ 320
 fir : 1.5 G[1] + 2 G[2] + 2 G[3] ≤ 720
 G[1] ≥ 0
 G[2] ≥ 0
 G[3] ≥ 0
```

Next, to optimize, use the `optimize!()` function:

In [28]:
```
optimize!(forest_model)
```

```
LP   has 2 rows; 3 cols; 6 nonzeros
Coefficient ranges:
  Matrix [5e-01, 2e+00]
  Cost   [2e+02, 4e+02]
  Bound  [0e+00, 0e+00]
  RHS    [3e+02, 7e+02]
Solving LP without presolve, or with basis, or unconstrained
Model status        : Optimal
Objective value     :  1.1200000000e+05
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.00
```

You should get confirmation that a solution was found; if one was not, there's a chance something was wrong with your model formulation.

To find the values of the decision variables, use `value()` (which can be broadcasted over variable arrays):

In [29]:
```
@show value.(G);
```

```
value.(G) = [352.0, 0.0, 96.0]
```

Similarly, `objective_value()` finds the optimal value of the objective:

In [30]:
```
@show objective_value(forest_model);
```

```
objective_value(forest_model) = 112000.0
```

Finally, we can find the dual values of the constraints with `shadow_price()`. Do this for the constraints in your model using the block below.

In [34]:
```
@show shadow_price(spruce);
@show shadow_price(fir);
```

```
shadow_price(spruce) = 80.0
shadow_price(fir) = 120.0
```

`JuMP` also lets you evaluate other expressions that you might be interested in based on the solutions. For example, you can use the following block to calculate the total amount of plywood the mill would produce under the optimal solution:

In [35]:
```
@expression(forest_model, total_plywood, sum(G))
@show value.(total_plywood);
```

```
value.(total_plywood) = 448.0
```

References

Put any consulted sources here, including classmates you worked with/who helped you.